

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

-----□□□□□□□-----



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN CÔNG NGHỆ JAVA
Chủ Đề : Xây dựng game thể loại phiêu lưu

SINH VIÊN THỰC HIỆN : NHÓM 5
GIÁO VIÊN HƯỚNG DẪN : ĐÀO THỊ LỆ THỦY

Mục lục

Lời mở đầu.....	5
Lời cảm ơn.....	6
I. Tổng quan về đề tài.....	7
1.1. Đôi nét về đề tài.....	7
1.2. Phương hướng và giải quyết đề tài.....	8
II. Các class của game.....	10
2.1. Mô hình tổng quát về các class của game.....	10
2.2. Các file đồ họa:.....	11
2.3. Các Class cụ thể.....	11
2.3.1. Packages interface.....	11
2.3.1.1. Class GameFrame.....	11
2.3.1.2. Class GamePanel.....	13
2.3.1.3. Class inputManager.....	14
2.3.2. Package effect.....	14
2.3.2.1. Class Animation:.....	14
2.3.2.2. Class FrameImage.....	17
2.3.2.3. Class CacheDataLoader:.....	17
2.3.3. Packages object.....	19
2.3.3.1. Class BackgroundMap.....	19

2.3.3.2. Class PhysicalMap.....	20
2.3.3.3 Class Bullet.....	22
2.3.3.4 Class BulletManager.....	23
2.3.3.5. Class Camera.....	24
2.3.3.6. Class GameObject.....	25
2.3.3.7 Class Gameworld.....	26
2.3.3.8. Class Human.....	28
2.3.3.9. Class RedEyeDevil.....	29
2.3.3.10. Class FinalBoss.....	32
2.3.3.11. Class Megaman.....	34
2.3.3.12. Class BlueFire:.....	38
2.3.3.13. Class RedEyeBullet:.....	39
2.3.3.14. Class RocketBullet.....	40
2.3.3.15. Class PaticularObject.....	40
2.3.3.16. Class ParticularObjectManager.....	44
III. Một số hình ảnh về game.....	46
IV. Kết luận.....	52
4.1. Kết luận.....	52
4.2. Tài liệu tham khảo:.....	52

Lời mở đầu

Trong bối cảnh ngày càng gia tăng của ngành công nghiệp game, việc lập trình và phát triển trò chơi đã trở thành một lĩnh vực hấp dẫn và đầy thách thức. Bài báo cáo này xin trình bày về quá trình lập trình một trò chơi phiêu lưu và những yếu tố quan trọng trong việc tạo ra một trải nghiệm phiêu lưu đáng nhớ cho người chơi.

Bài báo cáo này tập trung vào việc lập trình trò chơi phiêu lưu, nơi người chơi khám phá, tìm hiểu và giải quyết các câu đố, và trải nghiệm một cốt truyện hấp dẫn. Bài báo cáo sẽ trình bày về quá trình thiết kế và triển khai một trò chơi phiêu lưu từ đầu đến cuối, bao gồm việc xác định cốt truyện, xây dựng thế giới game, tạo ra các nhân vật và quái vật, cũng như xử lý các yếu tố gameplay như di chuyển, tương tác

Ngoài ra, báo cáo cũng sẽ trình bày về các ngôn ngữ lập trình và công cụ phát triển phổ biến được sử dụng trong lĩnh vực lập trình game phiêu lưu. Nhóm sẽ sử dụng ngôn ngữ lập trình JAVA để phát triển trò chơi phiêu lưu.

Bài báo cáo này nhằm mục đích trình bày thông tin chi tiết và hướng dẫn về quá trình lập trình một trò chơi phiêu lưu, từ việc xác định các yếu tố cốt truyện cho đến triển khai và kiểm thử. Hy vọng rằng thông qua bài báo cáo này, mọi người có thể có cái nhìn sâu sắc hơn về quá trình lập trình game phiêu lưu và khám phá tiềm năng sáng tạo của nó trong việc tạo ra những trò chơi hấp dẫn và truyền cảm hứng cho người chơi

Lời cảm ơn

Cuối cùng, xin bày tỏ lòng biết ơn sâu sắc đến tất cả những người đã đóng góp vào quá trình hoàn thành bài báo cáo này. Nhóm em xin gửi lời cảm ơn đến giảng viên hướng dẫn và các thành viên trong nhóm nghiên cứu, những người đã cung cấp sự hỗ trợ, kiến thức và những ý kiến quý giá để nhóm em có thể hoàn thành bài báo cáo này.

Nhóm cũng muốn bày tỏ lòng biết ơn đến cộng đồng lập trình game và những nhà phát triển trò chơi phiêu lưu đã tạo ra những tác phẩm đáng kinh ngạc, làm say mê và gợi cảm hứng. Các công trình của họ đã đóng góp rất lớn vào sự phát triển và tiến bộ của ngành công nghiệp game.

Cuối cùng, xin gửi lời cảm ơn chân thành đến bạn đọc và những người quan tâm đến bài báo cáo này. Hy vọng rằng nội dung của bài báo cáo sẽ cung cấp cho bạn cái nhìn tổng quan và sự hiểu biết sâu sắc về quá trình lập trình game phiêu lưu. Hy vọng rằng bài báo cáo này sẽ góp phần vào việc thúc đẩy sự phát triển tiếp tục của ngành công nghiệp game và tạo ra những trò chơi phiêu lưu tuyệt vời trong tương lai.

Xin chân thành cảm ơn!

I. Tổng quan về đề tài

1.1. Đôi nét về đề tài

a, Giới thiệu :

Trong bài tập lớn này, chúng ta sẽ khám phá quá trình xây dựng một trò chơi thể loại phiêu lưu đầy mạo hiểm và bí ẩn megaman. Trò chơi sẽ mang đến cho người chơi một trải nghiệm hấp dẫn và thách thức, nơi họ có thể nhập vai vào vai trò của một nhân vật anh hùng và trải qua các cuộc phiêu lưu đầy kịch tính và hấp dẫn.

b,Mục tiêu:

Mục tiêu của dự án này là xây dựng một trò chơi phiêu lưu độc đáo, sử dụng các yếu tố như cốt truyện, đồ họa, âm thanh và gameplay để tạo ra một trải nghiệm tuyệt vời cho người chơi. Người chơi sẽ phải đánh bại quái vật và tương tác với các nhân vật chính megaman, đánh bại trùm cuối để tiến xa trong cuộc phiêu lưu.

c. Ý tưởng:

Ý tưởng bài tập lớn của nhóm là tạo ra một game thể loại phiêu lưu với các tính năng chính:

Thể giới phiêu lưu: Xây dựng một thế giới rộng lớn với môi trường chính là một nhà máy sản xuất robot

Cốt truyện hấp dẫn: Tạo ra một câu chuyện hấp dẫn và đầy kịch tính, với câu chuyện chính là cuộc phiêu lưu của Megaman trong cuộc đối đầu với những con robot nguy hiểm

Hành động và chiến đấu: Cung cấp hệ thống hành động đa dạng, cho phép người chơi chiến đấu với quái vật, các người máy và đối mặt với các trận đấu với trùm cuối đầy kịch tính..

d. Mô tả:

Game megaman cần phải có:

Khởi tạo trò chơi: là một Frame cho phép người chơi biết một số thông tin cần thiết, các thức điều khiển trò chơi.

Người điều khiển nhân vật : người chơi sử dụng máy tính để điều khiển nhân vật chính.

Map: nơi nhân vật tham gia thám hiểm. Ở đây gồm có nhân vật của người chơi điều khiển, các quái vật, trùm cuối mà nhân vật chính cần phải đánh bại

Thanh máu và số mạng còn lại: hiển thị số máu còn lại khi bị quái đánh trúng hoặc va chạm với quái vật

Chương ngại vật địa hình: một điều không thể thiếu khi nói đến trò chơi phiêu lưu, nơi người chơi phải dùng nút di chuyển để vượt qua địa hình

1.2. Phương hướng và giải quyết đề tài

a, Công nghệ sử dụng

Trong game megaman, chúng ta sẽ sử dụng các công nghệ sau:

Ngôn ngữ lập trình: Chúng ta sẽ sử dụng Java để xây dựng trò chơi. Java cung cấp một cú pháp dễ hiểu, đáng tin cậy và hỗ trợ mạnh mẽ cho phát triển game.

Thư viện đồ họa: Chúng ta có thể sử dụng các thư viện đồ họa như JavaFX hoặc LibGDX để tạo ra giao diện đồ họa cho trò chơi. Những thư viện này cung cấp các công cụ và lớp hỗ trợ để vẽ đồ họa, xử lý sự kiện và quản lý giao diện người dùng.

Quản lý tài nguyên: Sử dụng một hệ thống quản lý tài nguyên để quản lý hình ảnh, âm thanh và các tài liệu khác cần thiết cho trò chơi.

Công cụ phát triển: Sử dụng IDE như Netbeans để phát triển mã nguồn và xây dựng trò chơi một cách hiệu quả. Các IDE này cung cấp các tính năng như gỡ lỗi, kiểm tra lỗi cú pháp và hỗ trợ tự động hoàn thành mã.

b. Kế hoạch thực hiện

Dưới đây là kế hoạch thực hiện để xây dựng trò chơi megaman:

Xác định yêu cầu: Xác định các yêu cầu chính của trò chơi, bao gồm cốt truyện, tính năng gameplay, đồ họa, âm thanh và giao diện người dùng.

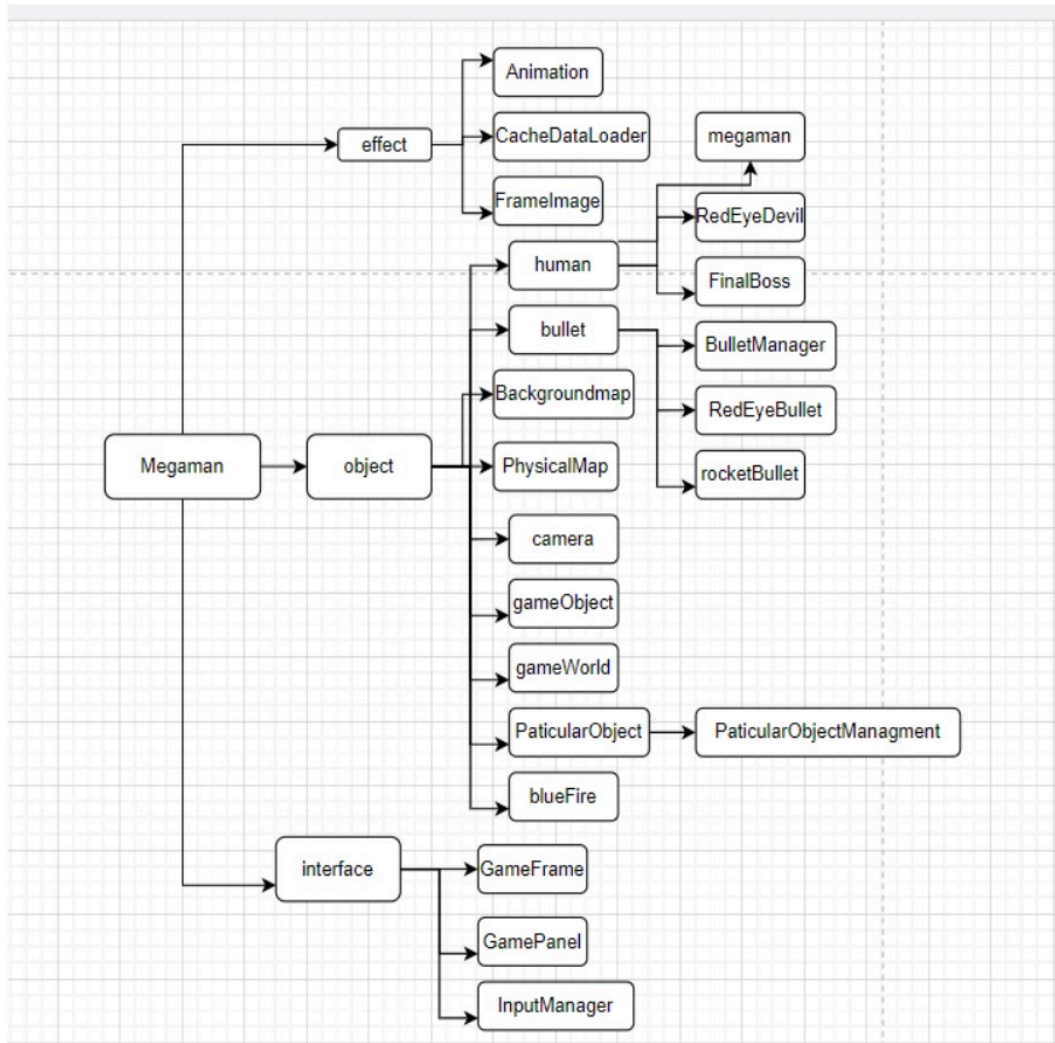
Thiết kế cấu trúc: Thiết kế cấu trúc tổ chức của trò chơi, bao gồm các lớp, giao diện và cơ chế xử lý sự kiện. Đảm bảo rằng trò chơi được tổ chức một cách rõ ràng và dễ bảo trì.

Xử lý đồ họa và âm thanh: Tạo ra các tài nguyên đồ họa và âm thanh cho trò chơi, bao gồm các file hình ảnh, hiệu ứng âm thanh và nhạc nền. Sử dụng các công cụ và thư viện để tải và hiển thị các tài nguyên này trong trò chơi.

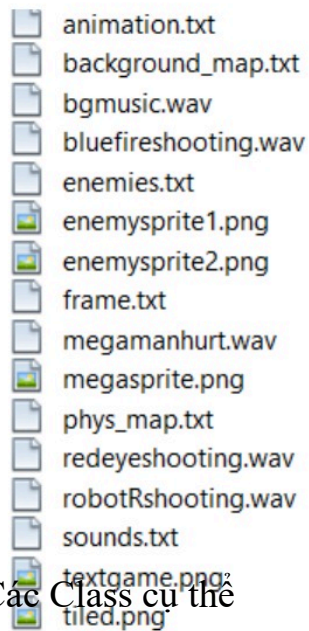
Xử lý sự kiện: Xử lý các sự kiện người dùng như phím bấm, chuột và cảm ứng để điều khiển nhân vật và tương tác với trò chơi. Đảm bảo rằng người chơi có thể di chuyển, tương tác và chiến đấu trong trò chơi một cách trơn tru.

II. Các class của game

2.1. Mô hình tổng quát về các class của game



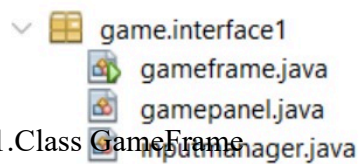
2.2.Các file đồ họa:



2.3. Các Class cụ thể

2.3.1.Packages interface

Bao gồm các class gameframe,gamepanel và inputmanager.Có chức năng tạo giao diện cũng như tạo frame cho game



2.3.1.1.Class GameFrame

Các phương thức chính:

```

package game.interfacel;
import java.awt.Dimension;
import java.awt.Toolkit;
import javax.swing.JFrame;

```

```

public class gameframe extends JFrame {
    public static final int SCREEN_WIDTH=1000;
    public static final int SCREEN_HEIGHT=600;

```

Ở đoạn code trên, lớp `gameframe` mở rộng từ lớp `JFrame`. Lớp `JFrame` là một lớp trong thư viện Swing của Java và được sử dụng để tạo ra một cửa sổ ứng dụng GUI.

Các thuộc tính `SCREEN_WIDTH` và `SCREEN_HEIGHT` được khai báo là hằng số và được gán giá trị 1000 và 600 tương ứng. Chúng định nghĩa kích thước của cửa sổ trò chơi.

```

public gameframe() {

    Toolkit toolkit = this.getToolkit(); //ĐỘ RỘNG MÀN HÌNH
    Dimension dimension = toolkit.getScreenSize();
    this.setBounds((dimension.width-SCREEN_WIDTH)/2, (dimension.height-SCREEN_HEIGHT)/2, SCREEN_WIDTH, SCREEN_HEIGHT);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    gamepanel=new gamepanel();
    add(gamepanel);
    this.addKeyListener(gamepanel);
}

```

Trong phương thức khởi tạo `gameframe()`, các bước sau được thực hiện:

`Toolkit toolkit = this.getToolkit();` Lấy đối tượng `Toolkit` để có thể truy cập đến các thông tin về hệ thống, bao gồm cả thông tin về độ rộng và chiều cao của màn hình.

`Dimension dimension = toolkit.getScreenSize();` Lấy kích thước của màn hình bằng cách sử dụng đối tượng `Toolkit` và phương thức `getScreenSize()`. Kích thước này được lưu trữ trong đối tượng `Dimension`.

this.setBounds((dimension.width-SCREEN_WIDTH)/2,(dimension.height-SCREEN_HEIGHT)/2, SCREEN_WIDTH, SCREEN_HEIGHT): Đặt vị trí và kích thước của cửa sổ. Vị trí ngang và dọc được tính bằng cách lấy trung bình cách viền màn hình và kích thước cửa sổ. Điều này giúp cửa sổ xuất hiện ở giữa màn hình. Kích thước của cửa sổ được đặt là SCREEN_WIDTH và SCREEN_HEIGHT, giá trị này đã được khai báo là 1000 và 600 tương ứng.

this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE): Đặt hành động mặc định khi người dùng đóng cửa sổ. Trong trường hợp này, khi cửa sổ được đóng, ứng dụng sẽ kết thúc và thoát.

gamepanel = new gamepanel(): Khởi tạo một đối tượng gamepanel, có thể là một lớp con của JPanel hoặc một lớp tùy chỉnh khác. Đối tượng này sẽ được sử dụng để hiển thị nội dung trò chơi trong cửa sổ.

add(gamepanel): Thêm đối tượng gamepanel vào cửa sổ, để hiển thị nội dung trò chơi.

this.addKeyListener(gamepanel): Đăng ký gamepanel như một bộ lắng nghe sự kiện phím. Điều này cho phép gamepanel nhận và xử lý các sự kiện phím từ người dùng.

2.3.1.2.Class GamePanel

Các phương thức chính:

Phương thức startGame() được giả định là một phương thức trong lớp gameframe và có nhiệm vụ khởi động trò chơi.

```
public void run() {
    long FPS=80;
    long period=1000*1000000/FPS;
    long begintime;
    long sleeptime;
    begintime= System.nanoTime();
    while(isRunning){
        long deltatime=System.nanoTime()- begintime;
        sleeptime= period-deltatime;

        try {
            if(sleeptime>0)
                Thread.sleep(sleeptime/1000000);
            else Thread.sleep(period/2000000);
        } catch (InterruptedException ex) {}

        begintime=System.nanoTime();
    }
}
```

Phương thức run() được giả định là một phương thức trong lớp gameframe và có chức năng điều khiển vòng lặp chạy trò chơi.

Phương thức run() này được sử dụng để duy trì tốc độ khung hình ổn định trong trò chơi bằng cách ngừng thực thi trong một khoảng thời gian nhất định giữa các khung hình.

Ba phương thức KeyType được sử dụng để xử lý các sự kiện phím trong trò chơi. Phương thức keyPressed(KeyEvent e) được gọi khi một phím được nhấn xuống, keyReleased(KeyEvent e) được gọi khi một phím được nhả ra, và keyTyped(KeyEvent e) được gọi khi một phím được gõ.

2.3.1.3. Class inputManager

Phương thức chính:

Lớp inputmanager định nghĩa một lớp đơn giản để xử lý sự kiện phím trong trò chơi. Trong lớp này, có một phương thức setPressedbtton(int keycode) để xử lý các sự kiện khi một phím được nhấn xuống, sau này sẽ là xử lý nhân vật di chuyển.

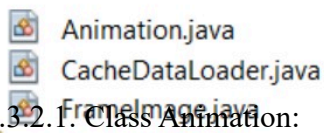
Các bước trong phương thức processKeypress(ed(int keycode) là như sau:

Trong mỗi trường hợp, sử dụng các hằng số từ lớp KeyEvent (ví dụ: KeyEvent.VK_UP, KeyEvent.VK_DOWN, vv.) để so sánh với keycode và xác định phím nào đã được nhấn xuống.

Tương tự với phương thức processKeypress(ed, phương thức processKeyrelease dùng để thông báo 1 phím nào đó được nhả ra

2.3.2. Package effect

Gồm 3 class: Animation, CacheDataLoader và FrameImage. Có chức năng tạo hiệu ứng cho nhân vật trong game



2.3.2.1. Class Animation:

Đoạn mã đầu tiên trong class định nghĩa và khởi tạo các biến thành viên của lớp Animation và thiết lập giá trị ban đầu cho chúng

```

public Animation(Animation animation){

    beginTime = animation.beginTime;
    currentFrame = animation.currentFrame;
    drawRectFrame = animation.drawRectFrame;
    isRepeated = animation.isRepeated;

    delayFrames = new ArrayList<Double>();
    for(Double d : animation.delayFrames){
        delayFrames.add(d);
    }

    ignoreFrames = new ArrayList<Boolean>();
    for(boolean b : animation.ignoreFrames){
        ignoreFrames.add(b);
    }

    frameImages = new ArrayList<FrameImage>();
    for(FrameImage f : animation.frameImages){
        frameImages.add(new FrameImage(f));
    }
}

```

Đoạn mã trên định nghĩa một constructor (hàm khởi tạo) cho lớp Animation nhận một đối tượng Animation khác làm đối số. Constructor này được sử dụng để tạo một đối tượng Animation mới dựa trên một đối tượng Animation đã tồn tại. Việc này cho phép sao chép tất cả các thuộc tính và danh sách khung hình từ đối tượng animation sang đối tượng hiện tại, tạo ra một bản sao độc lập của animation ban đầu.

Trong class này, có một số phương thức get và set các thuộc tính của đối tượng Animation. Các phương thức này cho phép ta thiết lập và truy xuất các thuộc tính của đối tượng Animation một cách dễ dàng và an toàn.

```

public void Update(long deltaTime) {
    if(beginTime == 0) beginTime = deltaTime;
    else{
        if(deltaTime - beginTime > delayFrames.get(currentFrame)){
            nextFrame();
            beginTime = deltaTime;
        }
    }
}

```

Ở đoạn code trên, phương thức Update được sử dụng để kiểm tra xem đã đến thời điểm chuyển khung hình tiếp theo trong animation hay chưa. Nếu đã đến, phương thức sẽ cập nhật currentFrame và đặt lại beginTime để chuẩn bị cho khung hình tiếp theo

```

public boolean isLastFrame() {
    if(currentFrame == frameImages.size() - 1)
        return true;
    else return false;
}

private void nextFrame() {
    if(currentFrame >= frameImages.size() - 1){
        if(isRepeated) currentFrame = 0;
    }
    else currentFrame++;

    if(ignoreFrames.get(currentFrame)) nextFrame();
}

```

Phương thức isLastFrame kiểm tra xem khung hình hiện tại có phải là khung hình cuối cùng trong danh sách frameImages hay không. Phương thức này trả về true nếu currentFrame bằng với kích thước của frameImages trừ 1, và false trong trường hợp khác.

Phương thức nextFrame được sử dụng để chuyển đến khung hình tiếp theo trong animation, xử lý việc lặp lại và bỏ qua các khung hình khi cần thiết.

```
public void draw(int x, int y, Graphics2D g2){  
  
    BufferedImage image = getCurrentImage();  
  
    g2.drawImage(image, x - image.getWidth()/2, y - image.getHeight()/2, null);  
    if(drawRectFrame)  
        g2.drawRect(x - image.getWidth()/2, x + image.getWidth()/2, image.getWidth(), image.getHeight());  
}
```

Phương thức draw được sử dụng để vẽ hình ảnh hiện tại lên một đối tượng Graphics2D (g2) tại vị trí (x, y). Ngoài ra, phương thức cũng có thể vẽ một hình chữ nhật xung quanh hình ảnh nếu drawRectFrame được đặt là true

2.3.2.2.Class FrameImage

```
public FrameImage(FrameImage frameImage){  
    image = new BufferedImage(frameImage.getWidthImage(),  
        frameImage.getHeightImage(), frameImage.image.getType());  
    Graphics g = image.getGraphics();  
    g.drawImage(frameImage.image, 0, 0, null);  
    name = frameImage.name;  
}
```

Phương thức FrameImage được sử dụng để tạo một bản sao của đối tượng FrameImage đã cho, bằng cách tạo một BufferedImage mới và sao chép hình ảnh từ frameImage vào đó.

```
public void draw(int x, int y, Graphics2D g2){  
  
    g2.drawImage(image, x - image.getWidth()/2, y - image.getHeight()/2, null);  
}
```

Phương thức draw được sử dụng để vẽ hình ảnh lên một đối tượng Graphics2D tại vị trí (x, y), với hình ảnh được căn giữa tại điểm (x, y).

Trong class này, cũng có các phương thức được sử dụng để truy xuất và gán thông tin về kích thước và tên của hình ảnh.

2.3.2.3. Class CacheDataLoader:

Trong class này, có các phương thức get,set được sử dụng để truy xuất đối tượng âm thanh (AudioClip), đối tượng hoạt hình (Animation) và đối tượng hình ảnh khung (FrameImage) từ đối tượng CacheDataLoader.

```
public void LoadSounds() throws IOException{
    sounds = new Hashtable<String, AudioClip>();

    FileReader fr = new FileReader(soundfile);
    BufferedReader br = new BufferedReader(fr);

    String line = null;

    if(br.readLine()==null) { // no line = "" or something like that
        System.out.println("No data");
        throw new IOException();
    }
    else {

        fr = new FileReader(soundfile);
```

Phương thức LoadSounds được sử dụng để tải và lưu trữ các âm thanh từ một tệp tin âm thanh. Phương thức này sẽ đọc và xử lý tệp tin âm thanh và lưu trữ các âm thanh vào Hashtable<String, AudioClip> có tên sounds trong đối tượng CacheDataLoader.

Tương tự, trong class này còn những phương thức khác giống với phương thức trên như:

Phương thức LoadBackgroundMap được sử dụng để đọc và lưu trữ thông tin về bản đồ nền từ tệp tin vào một mảng hai chiều trong đối tượng CacheDataLoader.

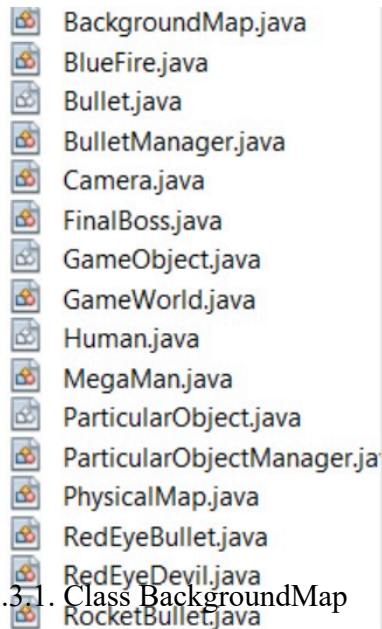
Phương thức LoadPhysMap được sử dụng để đọc và lưu trữ thông tin về bản đồ vật lý từ tệp tin vào một mảng hai chiều trong đối tượng CacheDataLoader.

Phương thức LoadAnimation được sử dụng để đọc và lưu trữ thông tin về các hoạt hình từ tệp tin vào một Hashtable trong đối tượng CacheDataLoader.

Phương thức LoadFrame được sử dụng để đọc và lưu trữ thông tin về các khung hình từ tệp tin vào một Hashtable trong đối tượng CacheDataLoader.

2.3.3. Packages object

Đây là Package quan trọng nhất, có chức năng tạo nhân vật, thử thách, quái, map và mọi thứ có trong game



```
public class BackgroundMap extends GameObject {  
  
    public int[][] map;  
    private int tileSize;  
  
    public BackgroundMap(float x, float y, GameWorldState gameWorld) {  
        super(x, y, gameWorld);  
        map = CacheDataLoader.getInstance().getBackgroundMap();  
        tileSize = 30;  
    }  
}
```

Lớp BackgroundMap là một đối tượng nền game được xây dựng từ thông tin bản đồ nền và có vị trí x, y cụ thể trong trạng thái thế giới game.

Trong phương thức khởi tạo, đầu tiên gọi phương thức khởi tạo của lớp cơ sở GameObject để thiết lập vị trí và trạng thái thế giới game của đối tượng nền.

Tiếp theo, gọi phương thức `getBackgroundMap` từ `CacheDataLoader.getInstance()` để lấy thông tin về bản đồ nền và gán cho thuộc tính `map` của đối tượng `BackgroundMap`

Cuối cùng, đặt giá trị của `tileSize` là 30.

```
public void draw(Graphics2D g2){  
  
    Camera camera = getGameWorld().camera;  
  
    g2.setColor(Color.RED);  
    for(int i = 0; i < map.length; i++)  
        for(int j = 0; j < map[0].length; j++)  
            if(map[i][j] != 0 && j*tileSize - camera.getPosX() > -30 && j*tileSize - camera.getPosX() < GameFrame.SCREEN_WIDTH  
                && i*tileSize - camera.getPosY() > -30 && i*tileSize - camera.getPosY() < GameFrame.SCREEN_HEIGHT){  
                g2.drawImage(CacheDataLoader.getInstance().getFrameImage("tiled"+map[i][j]).getImage(), (int) getPosX() + j*tileSize - (i  
                    (int) getPosY() + i*tileSize - (int) camera.getPosY(), null);  
            }  
}
```

Phương thức `draw` được sử dụng để vẽ bản đồ nền lên đối tượng `Graphics2D` được cung cấp, được sử dụng để vẽ bản đồ nền lên đối tượng `Graphics2D`. Vẽ chỉ xảy ra cho các ô trong bản đồ có giá trị khác 0 và nằm trong phạm vi hiển thị của camera.

2.3.3.2. Class `PhysicalMap`

```
public Rectangle haveCollisionWithTop(Rectangle rect){  
  
    int posX1 = rect.x/tileSize;  
    posX1 -= 2;  
    int posX2 = (rect.x + rect.width)/tileSize;  
    posX2 += 2;  
  
    //int posY = (rect.y + rect.height)/tileSize;  
    int posY = rect.y/tileSize;  
  
    if(posX1 < 0) posX1 = 0;  
  
    if(posX2 >= phys_map[0].length) posX2 = phys_map[0].length - 1;  
  
    for(int y = posY; y >= 0; y--){  
        for(int x = posX1; x <= posX2; x++){  
            if(phys_map[y][x] == 1){  
                Rectangle r = new Rectangle((int) getPosX() + x * tileSize, (int) getPosY() + y * tileSize, tileSize, tileSize);  
                if(rect.intersects(r))  
                    return r;  
            }  
        }  
    }  
    return null;  
}
```

Phương thức `haveCollisionWithTop` được sử dụng để kiểm tra va chạm của một vật thể với các ô phía trên trong bản đồ vật lý và trả về hình chữ nhật đại diện cho ô va chạm (nếu có).

Tương tự, trong class này còn những phương thức khác giống với phương thức trên như:

-Phương thức `haveCollisionWithLand` được sử dụng để kiểm tra va chạm của một vật thể với các ô phía dưới (đất) trong bản đồ vật lý và trả về vật thể đại diện cho ô va chạm (nếu có), hoặc null nếu không có va chạm.

-Phương thức `haveCollisionWithLand` được sử dụng để kiểm tra va chạm của một vật thể với các ô phía dưới (đất) trong bản đồ vật lý và trả về vật thể đại diện cho ô va chạm (nếu có), hoặc null nếu không có va chạm.

-Phương thức `haveCollisionWithRightWall` được sử dụng để kiểm tra va chạm của một vật thể với các ô tường bên phải (right wall) trong bản đồ vật lý và trả về vật thể đại diện cho ô va chạm (nếu có), hoặc null nếu không có va chạm.

-Phương thức `haveCollisionLeftWall` được sử dụng để kiểm tra va chạm của một vật thể với các ô tường bên trái (left wall) trong bản đồ vật lý và trả về vật thể đại diện cho ô va chạm (nếu có), hoặc null nếu không có va chạm.

```
public void draw(Graphics2D g2){  
    Camera camera = getGameWorld().camera;  
  
    g2.setColor(Color.GRAY);  
    for(int i = 0; i < phys_map.length; i++)  
        for(int j = 0; j < phys_map[0].length; j++)  
            if(phys_map[i][j] != 0) g2.fillRect((int) getPosX() + j*tileSize - (int) camera.getPosX(),  
                (int) getPosY() + i*tileSize - (int) camera.getPosY(), tileSize, tileSize);  
}
```

Phương thức `draw` được sử dụng để vẽ Physical map lên màn hình.

Các bước trong phương thức `draw` là như sau:

Lấy đối tượng Camera từ `getGameWorld().camera` để biết vị trí hiện tại của camera.

Sử dụng hai vòng lặp for lồng nhau để duyệt qua các phần tử trong mảng `phys_map`.

Trong mỗi vòng lặp, kiểm tra xem giá trị của `phys_map[i][j]` có khác 0 hay không. Nếu khác 0, sử dụng `g2.fillRect` để vẽ một hình chữ nhật tại vị trí `(getPosX() + j*tileSize - (int) camera.getPosX(), getPosY() + i*tileSize - (int) camera.getPosY())` với kích thước là `tileSize`.

Đây là 1 phần của physical map:

Cập nhật vị trí của đạn bằng cách tăng giá trị x của vị trí hiện tại (`getPosX()`) với giá trị tốc độ x (`getSpeedX()`) và tăng giá trị y của vị trí hiện tại (`getPosY()`) với giá trị tốc độ y (`getSpeedY()`). Điều này sẽ di chuyển đạn theo hướng và vận tốc đã được thiết lập trước đó.

Kiểm tra va chạm của đạn với đối tượng kẻ địch bằng cách sử dụng phương thức `getCollisionWidthEnemyObject(this)`. Kết quả trả về là một đối tượng `ParticularObject` (đối tượng kẻ địch) hoặc null nếu không có va chạm xảy ra.

Nếu đối tượng va chạm (object) khác null và trạng thái của đối tượng là `ALIVE`, thực hiện các hành động sau:

Đặt giá trị máu (`setBlood`) của đạn về 0, có thể là để đánh dấu đạn đã bị tiêu diệt.

Gọi phương thức `beHurt(getDamage())` của đối tượng kẻ địch để gây sát thương cho nó với mức sát thương của đạn.

2.3.3.4 Class BulletManager

```
public void UpdateObjects() {  
    super.UpdateObjects();  
    synchronized(particularObjects){  
        for(int id = 0; id < particularObjects.size(); id++){  
  
            ParticularObject object = particularObjects.get(id);  
  
            if(object.isObjectOutOfCameraView() || object.getState() == ParticularObject.DEATH){  
                particularObjects.remove(id);  
            }  
        }  
    }  
}
```

Phương thức `UpdateObjects` được sử dụng để cập nhật trạng thái của các đối tượng cụ thể (particular objects) trong trò chơi.

Sử dụng từ khóa `synchronized` để đồng bộ hóa việc truy cập vào danh sách `particularObjects`. Điều này đảm bảo rằng không có luồng khác có thể thay đổi danh sách này trong quá trình lặp.

Trong mỗi vòng lặp, lấy đối tượng cụ thể (`ParticularObject`) tại vị trí `id` từ danh sách `particularObjects`.

Kiểm tra hai điều kiện:

isObjectOutOfCameraView(): Kiểm tra xem đối tượng có nằm ngoài tầm nhìn của camera không. Nếu đúng, đối tượng được coi là không cần thiết và sẽ bị xóa khỏi danh sách.

getState() == ParticularObject.DEATH: Kiểm tra trạng thái của đối tượng có là DEATH hay không. Nếu đúng, đối tượng được coi là đã bị tiêu diệt và sẽ bị xóa khỏi danh sách.

Nếu một trong hai điều kiện trên được thỏa mãn, sử dụng phương thức remove(id) để xóa đối tượng tại vị trí id khỏi danh sách particularObjects.

2.3.3.5. Class Camera

Phương thức chính:

```
public void Update() {  
    if(!isLocked){  
  
        MegaMan mainCharacter = getGameWorld().megaMan;  
  
        if(mainCharacter.getPosX() - getPosX() > 400) setPosX(mainCharacter.getPosX() - 400);  
        if(mainCharacter.getPosX() - getPosX() < 200) setPosX(mainCharacter.getPosX() - 200);  
  
        if(mainCharacter.getPosY() - getPosY() > 400) setPosY(mainCharacter.getPosY() - 400); // bottom  
        else if(mainCharacter.getPosY() - getPosY() < 250) setPosY(mainCharacter.getPosY() - 250); // top  
    }  
}
```

Phương thức Update được sử dụng để cập nhật vị trí của camera trong trò chơi.

Các bước trong phương thức Update là như sau:

Kiểm tra biến isLocked. Nếu isLocked là false, tức là camera chưa bị khóa, thực hiện các bước sau:

Lấy tham chiếu đến đối tượng MegaMan (nhân vật chính) từ trạng thái thế giới trò chơi thông qua getGameWorld().megaMan.

Kiểm tra vị trí x của nhân vật chính và vị trí x của camera. Nếu hiệu của hai vị trí này lớn hơn 400, đặt vị trí x của camera là vị trí x của nhân vật chính trừ đi 400. Mục đích của bước này là giữ cho nhân vật chính luôn nằm trong phạm vi trung tâm của camera.

Kiểm tra vị trí y của nhân vật chính và vị trí y của camera. Nếu hiệu của hai vị trí này nhỏ hơn 200, đặt vị trí y của camera là vị trí y của nhân vật chính trừ đi 200. Mục đích của bước này là giữ cho nhân vật chính luôn nằm trong phạm vi trung tâm của camera.

Kiểm tra vị trí y của nhân vật chính và vị trí y của camera. Nếu hiệu của hai vị trí này lớn hơn 400, đặt vị trí y của camera là vị trí y của nhân vật chính trừ đi 400. Mục đích của bước này là giữ cho nhân vật chính luôn nằm trong phạm vi trung tâm của camera ở phía dưới.

Ngược lại, nếu hiệu của hai vị trí này nhỏ hơn 250, đặt vị trí y của camera là vị trí y của nhân vật chính trừ đi 250. Mục đích của bước này là giữ cho nhân vật chính luôn nằm trong phạm vi trung tâm của camera ở phía trên.

Tổng quan, phương thức Update được sử dụng để cập nhật vị trí của camera trong trò chơi dựa trên vị trí của nhân vật chính (MegaMan). Camera sẽ di chuyển để giữ cho nhân vật chính luôn nằm trong phạm vi trung tâm của camera, đồng thời giới hạn phạm vi di chuyển theo chiều dọc. Biến isLocked được sử dụng để kiểm soát việc khóa camera

2.3.3.6. Class GameObject

```
public abstract class GameObject {  
  
    private float posX;  
    private float posY;  
  
    private GameWorldState gameWorld;  
  
    public GameObject(float x, float y, GameWorldState gameWorld){  
        posX = x;  
        posY = y;  
        this.gameWorld = gameWorld;  
    }  
  
    public void setPosX(float x){  
        posX = x;  
    }  
  
    public float getPosX(){  
        return posX;  
    }  
}
```

Lớp GameObject là một lớp trừu tượng chứa các thuộc tính và phương thức cơ bản cho tất cả các đối tượng trong trò chơi. Lớp này cung cấp các phương thức để quản lý vị trí của đối tượng và truy xuất đến trạng thái thể giới trò chơi. Các lớp con của GameObject sẽ triển khai phương thức Update() để cập nhật trạng thái của đối tượng trong trò chơi. Lớp này chứa các phương thức set,get để thiết lập và truy xuất các vị trí (x,y) của các đối tượng trong trò chơi

2.3.3.7 Class Gameworld

Phương thức `initEnemies` được sử dụng để khởi tạo các đối tượng kẻ thù trong trò chơi. Dưới đây là các bước trong phương thức `initEnemies`:

Khởi tạo một đối tượng redeye kiểu `RedEyeDevil` với vị trí ban đầu (1250, 410) và tham chiếu đến trạng thái thế giới trò chơi (`this`). Đối tượng redeye là một kẻ thù loại `RedEyeDevil`.

Đặt hướng di chuyển của redeye thành `LEFT_DIR` (hướng trái).

Đặt loại đội hình của redeye thành `ENEMY_TEAM` (đội hình kẻ thù). Thêm redeye vào `particularObjectManager` thông qua phương thức `addObject(redeye)`.

Tương tự như vậy, ta cũng sẽ tạo được các loại kẻ thù khác

```
private void TutorialUpdate() {
    switch(tutorialState) {
        case INTROGAME:
            if (storyTutorial == 0) {
                if (openIntroGameY < 450) {
                    openIntroGameY += 4;
                } else storyTutorial++;
            } else {
                if (currentSize < textTutorial.length()) currentSize++;
            }
            break;
        case MEETFINALBOSS:
            if (storyTutorial == 0) {
                if (openIntroGameY >= 450) {
                    openIntroGameY -= 1;
                }
                if (camera.getPosX() < finalBossX) {
                    camera.setPosX(camera.getPosX() + 2);
                }
            }
            if (megaMan.getPosX() < finalBossX + 150) {
                megaMan.setDirection(ParticularObject.RIGHT_DIR);
            }
    }
}
```

Phương thức `TutorialUpdate` cập nhật trạng thái của hướng dẫn trong trò chơi dựa trên giá trị của `tutorialState` và `storyTutorial`. Nó thay đổi các giá trị của các biến và thực hiện các hành động như di chuyển camera, di chuyển nhân vật chính và thay đổi giá trị trong bản đồ để tạo hiệu ứng hướng dẫn cho người chơi.

```

public void Update() {
    switch(state){
        case INIT_GAME:
            break;
        case TUTORIAL:
            TutorialUpdate();
            break;
        case GAMEPLAY:
            particularObjectManager.UpdateObjects();
            bulletManager.UpdateObjects();

            physicalMap.Update();
            camera.Update();

            if(megaMan.getPosX() > finalBossX && finalbossTrigger){
                finalbossTrigger = false;
                switchState(TUTORIAL);
                tutorialState = MEETFINALBOSS;
                storyTutorial = 0;
            }
    }
}

```

Phương thức Update cập nhật trạng thái của trò chơi dựa trên giá trị của biến state. Nó cập nhật trạng thái của các đối tượng, bản đồ vật lý và camera trong trò chơi, và kiểm tra các điều kiện để chuyển trạng thái và thực hiện hành động tương ứng.

```

public void Render() {
    Graphics2D g2 = (Graphics2D) bufferedImage.getGraphics();

    if(g2!=null){
        switch(state){
            case INIT_GAME:
                g2.setColor(Color.BLACK);
                g2.fillRect(0, 0, GameFrame.SCREEN_WIDTH, GameFrame.SCREEN_HEIGHT);
                g2.setColor(Color.WHITE);
                g2.drawString("PRESS ENTER TO CONTINUE", 400, 300);
                break;
            case PAUSEGAME:
                g2.setColor(Color.BLACK);
                g2.fillRect(300, 260, 500, 70);
                g2.setColor(Color.WHITE);
                g2.drawString("PRESS ENTER TO CONTINUE", 400, 300);
                break;
            case TUTORIAL:
                // TUTORIAL
        }
    }
}

```

Phương thức Render sử dụng đối tượng Graphics2D để vẽ các đối tượng và giao diện người dùng tương ứng với trạng thái của trò chơi, bao gồm các giao diện:

Giao diện TUTORIAL, YOUWIN, GAMEOVER,

2.3.3.8. Class Human

```
public void Update(){
    super.Update();

    if(getState() == ALIVE || getState() == NOBEHURT){
        if(!isLanding){
            setPosX(getPosX() + getSpeedX());

            if(getDirection() == LEFT_DIR &&
                getGameWorld().physicalMap.haveCollisionWithLeftWall(getBoundForCollisionWithMap())!=null){
                Rectangle rectLeftWall = getGameWorld().physicalMap.haveCollisionWithLeftWall(getBoundForCollisionWithMap());
                setPosX(rectLeftWall.x + rectLeftWall.width + getWidth()/2);
            }

            if(getDirection() == RIGHT_DIR &&
                getGameWorld().physicalMap.haveCollisionWithRightWall(getBoundForCollisionWithMap())!=null){
                Rectangle rectRightWall = getGameWorld().physicalMap.haveCollisionWithRightWall(getBoundForCollisionWithMap());
                setPosX(rectRightWall.x - getWidth()/2);
            }
        }
    }
}
```

Phương thức Update được sử dụng để cập nhật trạng thái của đối tượng. Dưới đây là các bước trong phương thức Update:

Kiểm tra trạng thái của đối tượng. Nếu trạng thái là ALIVE hoặc NOBEHURT, tiếp tục xử lý các bước tiếp theo. Nếu không, không có hành động cụ thể.

Kiểm tra xem đối tượng có đang ở trạng thái không đứng trên mặt đất (isLanding) hay không. Nếu không đứng trên mặt đất, tiếp tục xử lý các bước tiếp theo. Nếu đang đứng trên mặt đất, không có hành động cụ thể.

Di chuyển đối tượng theo tốc độ ngang (speedX). Tăng hoặc giảm giá trị posX của đối tượng dựa trên speedX.

Nếu hướng di chuyển của đối tượng là LEFT_DIR (trái) và đối tượng va chạm với tường bên trái, thực hiện các bước sau:

Lấy vật va chạm với tường bên trái (rectLeftWall) bằng cách gọi phương thức haveCollisionWithLeftWall của physicalMap với boundForCollisionWithMap của đối tượng.

Thiết lập posX của đối tượng bằng `rectLeftWall.x + rectLeftWall.width + getWidth()/2`. Điều này đảm bảo đối tượng không xuyên qua tường bên trái và giữ vị trí chính giữa đối tượng.

Nếu hướng di chuyển của đối tượng là `RIGHT_DIR` (phải) và đối tượng va chạm với tường bên phải, thực hiện các bước sau:

Lấy hình chữ nhật va chạm với tường bên phải (`rectRightWall`) bằng cách gọi phương thức `haveCollisionWithRightWall` của `physicalMap` với `boundForCollisionWithMap` của đối tượng.

Thiết lập posX của đối tượng bằng `rectRightWall.x - getWidth()/2`. Điều này đảm bảo đối tượng không xuyên qua tường bên phải và giữ vị trí chính giữa đối tượng.

Tóm lại, phương thức `Update` được sử dụng để cập nhật vị trí của đối tượng dựa trên tốc độ ngang và xử lý va chạm với các tường bên trái và bên phải.

Tương tự như vậy, phần tiếp theo của phương thức `update` sẽ là cập nhật vị trí của đối tượng dựa trên tốc độ và xử lý va chạm với mặt đất và đỉnh đầu.

2.3.3.9. Class `RedEyeDevil`

```
public RedEyeDevil(float x, float y, GameWorldState gameWorld) {  
    super(x, y, 127, 89, 0, 100, gameWorld);  
    backAnim = CacheDataLoader.getInstance().getAnimation("redeye");  
    forwardAnim = CacheDataLoader.getInstance().getAnimation("redeye");  
    forwardAnim.flipAllImage();  
    startTimeToShoot = 0;  
    setDamage(10);  
    setTimeForNoBehurt(300000000);  
    shooting = CacheDataLoader.getInstance().getSound("redeshooting");  
}
```

Constructor này khởi tạo một đối tượng `RedEyeDevil` với các thông số và giá trị ban đầu cần thiết, bao gồm vị trí, kích thước, hướng di chuyển, máu, animation và âm thanh.

```

public void attack() {

    shooting.play();
    Bullet bullet = new RedEyeBullet(getPosX(), getPosY(), getGameWorld());
    if(getDirection() == LEFT_DIR) bullet.setSpeedX(-8);
    else bullet.setSpeedX(8);
    bullet.setTeamType(getTeamType());
    getGameWorld().bulletManager.addObject(bullet);

}

```

```

public void Update() {
    super.Update();
    if(System.nanoTime() - startTimeToShoot > 1000*100000000){
        attack();
        System.out.println("Red Eye attack");
        startTimeToShoot = System.nanoTime();
    }
}

```

Phương thức attack được sử dụng để thực hiện hành động tấn công của đối tượng RedEyeDevil, bao gồm:

Gọi phương thức play() trên đối tượng shooting để phát âm thanh khi đối tượng RedEyeDevil bắn đạn.

Kiểm tra hướng di chuyển của RedEyeDevil bằng cách so sánh getDirection() với LEFT_DIR. Nếu hướng di chuyển là LEFT_DIR, thiết lập speedX của đạn thành -8 (di chuyển sang trái). Ngược lại, thiết lập speedX của đạn thành 8 (di chuyển sang phải).

Thiết lập teamType của đạn bằng getTeamType() của RedEyeDevil. Điều này xác định loại đội của đạn, để xử lý va chạm giữa các đối tượng.

Sau đó là phương thức Update được sử dụng để cập nhật trạng thái của RedEyeDevil theo từng khung hình

```

public void draw(Graphics2D g2) {
    if(!isObjectOutOfCameraView()){
        if(getState() == NOBEHURT && (System.nanoTime()/100000000)%2!=1){
            // plash...
        }else{
            if(getDirection() == LEFT_DIR){
                backAnim.Update(System.nanoTime());
                backAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()),
                    (int) (getPosY() - getGameWorld().camera.getPosY()), g2);
            }else{
                forwardAnim.Update(System.nanoTime());
                forwardAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()),
                    (int) (getPosY() - getGameWorld().camera.getPosY()), g2);
            }
        }
    }
}

```

Phương thức draw được sử dụng để vẽ đối tượng RedEyeDevil lên màn hình. Dưới đây là các bước trong phương thức:

Kiểm tra xem đối tượng RedEyeDevil có nằm ngoài tầm nhìn của camera không. Nếu đối tượng nằm ngoài tầm nhìn, không vẽ gì cả và kết thúc

Kiểm tra trạng thái của RedEyeDevil. Nếu RedEyeDevil đang trong trạng thái NOBEHURT và thời gian hiện tại không nằm trong khoảng 0.1 giây, không vẽ gì cả để tạo hiệu ứng nhấp nháy.

Nếu không nằm trong trạng thái NOBEHURT hoặc thời gian hiện tại nằm trong khoảng 0.1 giây, kiểm tra hướng di chuyển của RedEyeDevil. Nếu hướng di chuyển là LEFT_DIR, cập nhật animation backAnim và vẽ hình ảnh của backAnim tại vị trí tính từ vị trí hiện tại của RedEyeDevil và vị trí của camera

Ngược lại, nếu hướng di chuyển không phải là LEFT_DIR, cập nhật animation và vẽ hình ảnh của forwardAnim tại vị trí tính từ vị trí hiện tại của RedEyeDevil và vị trí của camera

Tóm lại, phương thức draw được sử dụng để vẽ đối tượng RedEyeDevil lên màn hình, dựa trên trạng thái và hướng di chuyển của đối tượng, và xử lý việc đối tượng nằm ngoài tầm nhìn của camera.

2.3.3.10. Class FinalBoss

```
public void Update(){
    super.Update();

    if(getGameWorld().megaMan.getPosX() > getPosX())
        setDirection(RIGHT_DIR);
    else setDirection(LEFT_DIR);

    if(startTimeForAttacked == 0)
        startTimeForAttacked = System.currentTimeMillis();
    else if(System.currentTimeMillis() - startTimeForAttacked > 300){
        attack();
        startTimeForAttacked = System.currentTimeMillis();
    }

    if(!attackType[attackIndex].equals("NONE")){
        if(attackType[attackIndex].equals("shooting")){

            Bullet bullet = new RocketBullet(getPosX(), getPosY() - 50, getGameWorld());
            if(getDirection() == LEFT_DIR) bullet.setSpeedX(-4);
            else bullet.setSpeedX(4);
            bullet.setTeamType(getTeamType());
        }
    }
}
```

Phương thức Update được sử dụng để cập nhật trạng thái của đối tượng Boss, dựa trên hành vi và tương tác với đối tượng MegaMan trong gameWorld. phương thức bao gồm:

Kiểm tra vị trí của MegaMan so với vị trí của Boss (để xác định hướng di chuyển của Boss. Nếu vị trí x của MegaMan lớn hơn vị trí x của Boss, thiết lập hướng di chuyển của Boss thành bên phải. Ngược lại, thiết lập hướng di chuyển thành trái.

Kiểm vị trí của boss, nếu là bên trái, thiết lập tốc độ đạn là -4 còn bên phải sẽ là 4, cùng với đó là thiết lập team của boss để đạn sẽ gây sát thương lên nhân vật

Tiếp theo, kiểm tra xem va chạm của boss với tường để thiết lập tốc độ di chuyển

Tổng quan, phương thức Update được sử dụng để cập nhật trạng thái và hành vi của Boss trong gameWorld, bao gồm xác định hướng di chuyển, thực hiện hành động tấn công và xử lý va chạm với tường.


```

@Override
public void attack() {

    if(System.currentTimeMillis() - lastAttackTime > timeAttack.get(attackType[attackIndex])){
        lastAttackTime = System.currentTimeMillis();

        attackIndex++;
        if(attackIndex >= attackType.length) attackIndex = 0;

        if(attackType[attackIndex].equals("slide")){
            if(getPosX() < getGameWorld().megaMan.getPosX()) setSpeedX(5);
            else setSpeedX(-5);
        }
    }
}

```

Phương thức attack được sử dụng để thực hiện hành động tấn công của Boss, bao gồm xác định thời gian tấn công và cập nhật trạng thái tấn công của Boss.

```

public void draw(Graphics2D g2) {

    if(getState() == NOBEHURT && (System.nanoTime()/100000000)%2!=1)
    {
        System.out.println("Flash...");
    }else{

        if(attackType[attackIndex].equals("NONE")){
            if(getDirection() == RIGHT_DIR){
                idleforward.Update(System.nanoTime());
                idleforward.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.g
            }else{
                idleback.Update(System.nanoTime());
                idleback.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getP
            }
        }else if(attackType[attackIndex].equals("shooting")){

            if(getDirection() == RIGHT_DIR){
                shootingforward.Update(System.nanoTime());
                shootingforward.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().came
            }else{
                shootingback.Update(System.nanoTime());
                shootingback.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.g
            }
        }
    }
}

```

Phương thức draw được sử dụng để vẽ Boss lên màn hình dựa trên trạng thái và loại tấn công hiện tại. Dưới đây là các bước trong phương thức:

Kiểm tra trạng thái của Boss có phải là NOBEHURT . Nếu điều kiện này đúng, hiển thị hiệu ứng nhấp nháy khi Boss đang trong trạng thái không thể bị thương.

Nếu điều kiện trên không đúng, tiếp tục vẽ Boss dựa trên loại tấn công hiện tại

Nếu boss không tấn công, kiểm tra hướng di chuyển của . Nếu hướng là RIGHT_DIR, cập nhật sprite của Boss trong trạng thái đứng yên với hướng đi về phía

trước . Ngược lại, cập nhật sprite của Boss trong trạng thái đứng yên với hướng đi về phía sau . Vẽ tương ứng lên màn hình.

Nếu boss đang bắn , kiểm tra hướng di chuyển của Boss. Tương tự như trên, cập nhật sprite của Boss trong trạng thái bắn với hướng đi về phía trước (shootingforward) hoặc phía sau (shootingback). Vẽ sprite tương ứng lên màn hình.

Nếu boss đang lướt, kiểm tra tốc độ di chuyển ngang của Boss . Nếu tốc độ lớn hơn 0, cập nhật Boss trong trạng thái trượt với hướng đi về phía trước (slideforward). Ngược lại, cập nhật sprite trong trạng thái trượt với hướng đi về phía sau (slideback). Vẽ tương ứng lên màn hình.

Tóm lại, phương thức draw được sử dụng để vẽ Boss lên màn hình dựa trên trạng thái và loại tấn công hiện tại của Boss

2.3.3.11.Class Megaman

```
runForwardAnim = CacheDataLoader.getInstance().getAnimation("run");
runBackAnim = CacheDataLoader.getInstance().getAnimation("run");
runBackAnim.flipAllImage();

idleForwardAnim = CacheDataLoader.getInstance().getAnimation("idle");
idleBackAnim = CacheDataLoader.getInstance().getAnimation("idle");
idleBackAnim.flipAllImage();

dickForwardAnim = CacheDataLoader.getInstance().getAnimation("dick");
dickBackAnim = CacheDataLoader.getInstance().getAnimation("dick");
dickBackAnim.flipAllImage();

flyForwardAnim = CacheDataLoader.getInstance().getAnimation("flyingup");
flyForwardAnim.setIsRepeated(false);
flyBackAnim = CacheDataLoader.getInstance().getAnimation("flyingup");
flyBackAnim.setIsRepeated(false);
flyBackAnim.flipAllImage();
```

Tất cả đoạn code trên tải và lưu trữ các hoạt ảnh từ CacheDataLoader vào các biến tương ứng, sẵn sàng để sử dụng trong việc vẽ các trạng thái và hành động của đối tượng trong game.

```

public Rectangle getBoundForCollisionWithEnemy() {
    Rectangle rect = getBoundForCollisionWithMap();

    if(getIsDicking()){
        rect.x = (int) getPosX() - 22;
        rect.y = (int) getPosY() - 20;
        rect.width = 44;
        rect.height = 65;
    }else{
        rect.x = (int) getPosX() - 22;
        rect.y = (int) getPosY() - 40;
        rect.width = 44;
        rect.height = 80;
    }

    return rect;
}

```

Phương thức `getBoundForCollisionWithEnemy()` trả về một hình chữ nhật (Rectangle) có các tọa độ (x,y) đại diện cho vùng va chạm của đối tượng với kẻ địch trong game

```

public void draw(Graphics2D g2) {
    switch(getState()){
        case ALIVE:
        case NOBEHURT:
            if(getState() == NOBEHURT && (System.nanoTime()/100000000)%2!=1)
            {
                System.out.println("Plash...");
            }else{
                if(getIsLanding()){
                    if(getDirection() == RIGHT_DIR){
                        landingForwardAnim.setCurrentFrame(landingBackAnim.getCurrentFrame());
                        landingForwardAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()),
                            (int) getPosY() - (int) getGameWorld().camera.getPosY() + (getBoundForColl
                            g2);
                    }else{
                        landingBackAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()),
                            (int) getPosY() - (int) getGameWorld().camera.getPosY() + (getBoundForColl

```

Phương thức `draw(Graphics2D g2)` được sử dụng để vẽ đối tượng trong game. Dưới đây là mô tả chi tiết của phương thức:

Trong trạng thái ALIVE hoặc NOBEHURT, kiểm tra nếu đối tượng đang trong trạng thái NOBEHURT và thời gian hiện tại thỏa mãn điều kiện, in ra chuỗi "Splash...". Nếu không, tiếp tục vẽ đối tượng.

Kiểm tra trạng thái isLanding() của đối tượng. Nếu đang ở trạng thái "đang hạ cánh", vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển của đối tượng.

Kiểm tra trạng thái isJumping() của đối tượng. Nếu đang trong trạng thái "đang nhảy", vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển của đối tượng. Nếu đang bắn, sử dụng các hình ảnh có liên quan.

Kiểm tra trạng thái isDicking() của đối tượng. Nếu đang trong trạng thái "đang ngồi", vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển của đối tượng.

Nếu không ở trong các trạng thái trên, kiểm tra tốc độ speedX của đối tượng. Nếu speedX lớn hơn 0, vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển của đối tượng khi đang chạy. Nếu speedX nhỏ hơn 0, vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển khi đối tượng đang chạy theo hướng ngược lại. Nếu speedX bằng 0, vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển khi đối tượng đang đứng yên.

Trong trạng thái BEHURT, vẽ các hình ảnh tương ứng với trạng thái và hướng di chuyển của đối tượng khi bị thương.

Trong trạng thái FEY, không có hành động nào được thực hiện.

```

public void jump() {
    if(!getIsJumping()){
        setIsJumping(true);
        setSpeedY(-5.0f);
        flyBackAnim.reset();
        flyForwardAnim.reset();
    }
    else{
        Rectangle rectRightWall = getBoundForCollisionWithMap();
        rectRightWall.x += 1;
        Rectangle rectLeftWall = getBoundForCollisionWithMap();
        rectLeftWall.x -= 1;

        if(getGameWorld().physicalMap.haveCollisionWithRightWall(rectRightWall)!=null && getSpeedX() > 0){
            setSpeedY(-5.0f);
            //setSpeedX(-1);
            flyBackAnim.reset();
            flyForwardAnim.reset();
            //setDirection(LEFT_DIR);
        }else if(getGameWorld().physicalMap.haveCollisionWithLeftWall(rectLeftWall)!=null && getSpeedX() < 0
        {
            setSpeedY(-5.0f);
            //setSpeedX(1);
            flyForwardAnim.reset();
            flyBackAnim.reset();
        }
    }
}

```

Phương thức `jump()` được sử dụng để xử lý hành động nhảy của đối tượng trong game.

Kiểm tra xem đối tượng có đang trong trạng thái "đang nhảy" hay không. Nếu không, thực hiện các bước tiếp theo để bắt đầu quá trình nhảy.

Đặt tốc độ theo chiều y (`speedY`) của đối tượng thành `-5.0f` để làm cho đối tượng nhảy lên trên.

Đặt lại các animation `flyBackAnim` và `flyForwardAnim` về trạng thái ban đầu bằng cách gọi phương thức `reset()`.

Kiểm tra xem đối tượng có đang tiếp xúc với tường bên phải hoặc tường bên trái không. Để làm điều này, tạo ra hai hình chữ nhật `rectRightWall` và `rectLeftWall` để kiểm tra va chạm với map.

Nếu đối tượng có va chạm với tường bên phải và đang di chuyển sang phải (`getSpeedX() > 0`), đặt tốc độ theo chiều y (`speedY`) của đối tượng thành `-5.0f` và đặt lại các animation.

Nếu đối tượng có va chạm với tường bên trái và đang di chuyển sang trái (`getSpeedX() < 0`), đặt tốc độ theo chiều y (`speedY`) của đối tượng thành `-5.0f` và đặt lại các animation.

```

@Override
public void attack() {

    if(!isShooting && !getIsDicking()){

        shooting1.play();

        Bullet bullet = new BlueFire(getPosX(), getPosY(), getGameW
        if(getDirection() == LEFT_DIR) {
            bullet.setSpeedX(-10);
            bullet.setPosX(bullet.getPosX() - 40);
            if(getSpeedX() != 0 && getSpeedY() == 0){
                bullet.setPosX(bullet.getPosX() - 10);
                bullet.setPosY(bullet.getPosY() - 5);
            }
        }else {
            bullet.setSpeedX(10);
            bullet.setPosX(bullet.getPosX() + 40);
            if(getSpeedX() != 0 && getSpeedY() == 0){
                bullet.setPosX(bullet.getPosX() + 10);
                bullet.setPosY(bullet.getPosY() - 5);
            }
        }
        if(getIsJumping())
            bullet.setPosY(bullet.getPosY() - 20);

        bullet.setTeamType(getTeamType());
        getGameWorld().add(bullet);
    }
}

```

Phương thức `attack()` được sử dụng để xử lý hành động tấn công của đối tượng trong game. Dưới đây là mô tả chi tiết của phương thức:

Kiểm tra xem đối tượng có đang trong trạng thái "đang bắn" (`isShooting`) hay không. Nếu không, thực hiện các bước tiếp theo để thực hiện hành động bắn.

Tạo một đối tượng bullet (đạn) mới dựa trên lớp `BlueFire`, với vị trí x và vị trí y của đối tượng hiện tại và truyền tham chiếu đến `gameWorld`

Nếu hướng di chuyển của đối tượng là `LEFT_DIR`, đặt tốc độ theo chiều x của đạn thành -10 và điều chỉnh vị trí x và y của đạn để nằm bên trái đối tượng. Nếu đối tượng đang di chuyển ngang (`getSpeedX() != 0`) và không đang di chuyển dọc (`getSpeedY() == 0`), điều chỉnh vị trí x và y của đạn một cách thích hợp.

Nếu hướng di chuyển (direction) của đối tượng là `RIGHT_DIR`, đặt tốc độ theo chiều x (speedX) của đạn thành 10 và điều chỉnh vị trí x và y của đạn để nằm bên phải đối tượng. Tương tự, nếu đối tượng đang di chuyển ngang và không di chuyển dọc, điều chỉnh vị trí x và y của đạn.

Nếu đối tượng đang trong trạng thái "đang nhảy" (`getIsJumping()`), điều chỉnh vị trí y của đạn để nằm cao hơn.

Đặt loại đội (teamType) của đạn bằng với loại đội của đối tượng hiện tại.

Thêm đạn vào quản lý đạn (bulletManager) của gameWorld bằng cách gọi phương thức addObject(bullet).

2.3.3.12. Class BlueFire:

```
public void draw(Graphics2D g2) {  
    // TODO Auto-generated method stub  
    if(getSpeedX() > 0){  
        if(!forwardBulletAnim.isIgnoreFrame(0) && forwardBulletAnim.getCurrentFrame() == 3){  
            forwardBulletAnim.setIgnoreFrame(0);  
            forwardBulletAnim.setIgnoreFrame(1);  
            forwardBulletAnim.setIgnoreFrame(2);  
        }  
  
        forwardBulletAnim.Update(System.nanoTime());  
        forwardBulletAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPos  
    }else{  
        if(!backBulletAnim.isIgnoreFrame(0) && backBulletAnim.getCurrentFrame() == 3){  
            backBulletAnim.setIgnoreFrame(0);  
            backBulletAnim.setIgnoreFrame(1);  
            backBulletAnim.setIgnoreFrame(2);  
        }  
  
        backBulletAnim.Update(System.nanoTime());  
        backBulletAnim.draw((int) (getPosX() + getGameWorld().camera.getPosX()), (int) getPos  
    }  
}
```

Đây là phương thức vẽ lên hình ảnh viên đạn đang bay với các điều kiện Speed

```
public void Update() {  
    // TODO Auto-generated method stub  
    if(forwardBulletAnim.isIgnoreFrame(0) || backBulletAnim.isIgnoreFrame(0))  
        setPosX(getPosX() + getSpeedX());  
    ParticularObject object = getGameWorld().particularObjectManager.getCollisionWidthEnemyObject(this);  
    if(object != null && object.getState() == ALIVE){  
        setBlood(0);  
        object.setBlood(object.getBlood() - getDamage());  
        object.setState(BEHURT);  
        System.out.println("Bullet set behurt for enemy");  
    }  
}
```

Phương thức Update() được sử dụng để cập nhật trạng thái của đối tượng trong game. Kiểm tra va chạm giữa đối tượng và các đối tượng kẻ thù (EnemyObject) trong particularObjectManager của gameWorld. Nếu có va chạm (object != null), và đối tượng kẻ thù đang trong trạng thái "sống" (object.getState() == ALIVE), thực hiện các thao tác sau đây:

Đặt mức máu (blood) của đối tượng hiện tại bằng 0 (setBlood(0)), tức là đối tượng đã chết.

Giảm mức máu (blood) của đối tượng kẻ thù đi mức sát thương của đối tượng hiện tại (object.getBlood() - getDamage()).

Đặt trạng thái của đối tượng kẻ thù thành "bị tổn thương" (BEHURT).

In ra thông báo "Bullet set behurt for enemy" trên cửa sổ console.

2.3.3.13. Class RedEyeBullet:

```
@Override
public void draw(Graphics2D g2) {
    // TODO Auto-generated method stub
    if(getSpeedX() > 0){
        forwardBulletAnim.Update(System.nanoTime());
        forwardBulletAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY(), g2);
    }else{
        backBulletAnim.Update(System.nanoTime());
        backBulletAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY(), g2);
    }
    //drawBoltForCollisionWithEnemy(g2);
}
```

Cũng như các phương thức về bullet ở trong class khác, ở class RedEyeBullet, Phương thức draw(Graphics2D g2) được sử dụng để vẽ đối tượng đạn lên màn hình trong game khi thỏa mãn điều kiện tốc độ của nhân vật(ở đây là quái RedEyeDevil)

Ngoài ra ở class này cũng có phương thức update như các class khác

2.3.3.14. Class RocketBullet

```
public void draw(Graphics2D g2) {
    // TODO Auto-generated method stub
    if(getSpeedX() > 0){
        if(getSpeedY() > 0){
            forwardBulletAnimDown.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPo
        }else if(getSpeedY() < 0){
            forwardBulletAnimUp.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY
        }else{
            forwardBulletAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY()
        }
    }else{
        if(getSpeedY() > 0){
            backBulletAnimDown.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY(
        }else if(getSpeedY() < 0){
            backBulletAnimUp.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY(),
        }else{
            backBulletAnim.draw((int) (getPosX() - getGameWorld().camera.getPosX()), (int) getPosY() - (int) getGameWorld().camera.getPosY(), g
    }
}
```

Cũng như các phương thức về bullet ở trong class khác, ở class RocketBullet, Phương thức draw(Graphics2D g2) được sử dụng để vẽ đối tượng đạn lên màn hình trong game khi thỏa mãn điều kiện tốc độ của nhân vật(ở đây là quái Boss)

Tuy nhiên ,Boss ở đây luôn di chuyển nên tọa độ của y sẽ thay đổi,nên sẽ có thêm phương thức changeSpeedY() dùng để thay đổi chiều viên đạn:

```
private void changeSpeedY() {
    if(System.currentTimeMillis() % 3 == 0){
        setSpeedY(getSpeedX());
    }else if(System.currentTimeMillis() % 3 == 1){
        setSpeedY(-getSpeedX());
    }else {
        setSpeedY(0);
    }
}
```

Phương thức `changeSpeedY()` được khai báo riêng và được gọi để thay đổi tốc độ theo chiều y của đối tượng. Nếu giá trị điều kiện thời gian được thỏa mãn (chia hết cho 3), tốc độ theo chiều y được đặt bằng tốc độ theo chiều x (`setSpeedY(getSpeedX())`). Nếu kết quả là 1, tốc độ theo chiều y được đặt bằng đối lưu của tốc độ theo chiều x (`setSpeedY(-getSpeedX())`). Nếu kết quả là 2, tốc độ theo chiều y được đặt bằng 0 (`setSpeedY(0)`).

2.3.3.15. Class ParticularObject

Đoạn đầu trong class này đa số là việc khởi tạo các giá trị như `alive`, `behurt`, `dead`,

Tiếp theo là các phương thức `set`, `get` dùng để thiết lập và truy xuất các đối tượng như máu, sát thương, tốc độ,

```
public boolean isObjectOutOfCameraView() {
    if (getPosX() - getGameWorld().camera.getPosX() > getGameWorld().camera.getWidthView() ||
        getPosX() - getGameWorld().camera.getPosX() < -50 ||
        getPosY() - getGameWorld().camera.getPosY() > getGameWorld().camera.getHeightView() ||
        getPosY() - getGameWorld().camera.getPosY() < -50)
        return true;
    else return false;
}
```

Phương thức `isObjectOutOfCameraView()` được sử dụng để kiểm tra xem đối tượng đạn có nằm ngoài tầm nhìn của camera không. Dưới đây là mô tả chi tiết của phương thức:

Tính toán khoảng cách giữa vị trí x của đối tượng đạn và vị trí x của camera (`getGameWorld().camera.getPosX()`). Nếu khoảng cách này lớn hơn chiều rộng của tầm nhìn của camera hoặc nhỏ hơn -50, tức là đối tượng đạn nằm ngoài phạm vi tầm nhìn theo chiều ngang, thì trả về `true` (đối tượng đạn nằm ngoài tầm nhìn).

Tương tự, tính toán khoảng cách giữa vị trí y của đối tượng đạn (`getPosY()`) và vị trí y của camera (`getGameWorld().camera.getPosY()`). Nếu khoảng cách này lớn hơn chiều cao của tầm nhìn của camera (`getGameWorld().camera.getHeightView()`), hoặc nhỏ hơn -

50, tức là đối tượng đạn nằm ngoài phạm vi tầm nhìn theo chiều dọc, thì trả về true (đối tượng đạn nằm ngoài tầm nhìn).

Nếu không thoả mãn bất kỳ điều kiện nào ở trên, tức là đối tượng đạn nằm trong phạm vi tầm nhìn của camera, thì trả về false (đối tượng đạn không nằm ngoài tầm nhìn).

```
public Rectangle getBoundForCollisionWithMap() {  
    Rectangle bound = new Rectangle();  
    bound.x = (int) (getPosX() - (getWidth()/2));  
    bound.y = (int) (getPosY() - (getHeight()/2));  
    bound.width = (int) getWidth();  
    bound.height = (int) getHeight();  
    return bound;  
}
```

Phương thức `getBoundForCollisionWithMap()` được sử dụng để lấy ranh giới va chạm của đối tượng đạn với các vật thể trong bản đồ. Dưới đây là mô tả chi tiết của phương thức:

Tạo một đối tượng hình chữ nhật (`Rectangle`) mới có tên là `bound`.

Đặt giá trị `x` của `bound` bằng cách lấy vị trí `x` của đối tượng đạn (`getPosX()`) và trừ đi nửa chiều rộng của đối tượng đạn (`getWidth()/2`). Điều này giúp đặt `bound.x` vào vị trí bên trái của đối tượng đạn.

Đặt giá trị `y` của `bound` bằng cách lấy vị trí `y` của đối tượng đạn (`getPosY()`) và trừ đi nửa chiều cao của đối tượng đạn (`getHeight()/2`). Điều này giúp đặt `bound.y` ở vị trí phía trên của đối tượng đạn.

Đặt chiều rộng của `bound` bằng giá trị của chiều rộng của đối tượng đạn (`getWidth()`).

Đặt chiều cao của `bound` bằng giá trị của chiều cao của đối tượng đạn (`getHeight()`).

Trả về đối tượng `bound` là ranh giới va chạm của đối tượng đạn với các vật thể trong bản đồ.

```

public void Update() {
    switch(state) {
        case ALIVE:
            // note: SET DAMAGE FOR OBJECT NO DAMAGE
            ParticularObject object = getGameWorld().particularObjectManager.getCollisionWidthEnemyObject(this);
            if(object!=null) {
                if(object.getDamage() > 0) {
                    // switch state to fey if object die
                    System.out.println("Fea Damage ... from collision with enemy ... " + object.getDamage());
                    beHurt(object.getDamage());
                }
            }
    }
}

```

Phương thức Update() có nhiệm vụ cập nhật trạng thái của đối tượng đạn theo từng khung hình. Dưới đây là mô tả chi tiết của phương thức:

Trường hợp ALIVE: Trong trạng thái sống, đối tượng đạn kiểm tra va chạm với đối tượng kẻ thù bằng cách gọi phương thức getCollisionWidthEnemyObject(this) từ particularObjectManager của gameWorld. Nếu có va chạm (object không null), kiểm tra nếu đối tượng kẻ thù có sát thương lớn hơn 0 (object.getDamage() > 0), thì trạng thái của đối tượng đạn chuyển sang BEHURT và gọi phương thức beHurt() để giảm máu tương ứng với sát thương.

Trường hợp BEHURT: Trong trạng thái bị thương, kiểm tra nếu behurtBackAnim là null (không có hình ảnh phản ứng khi bị thương), thì chuyển trạng thái sang NOBEHURT và đặt thời gian bắt đầu không bị thương (startTimeNoBeHurt) là thời điểm hiện tại. Nếu behurtBackAnim khác null, thì cập nhật hình ảnh của trạng thái bị thương bằng cách gọi phương thức Update(System.nanoTime()) của behurtForwardAnim. Nếu đã đến khung cuối cùng của hình ảnh (behurtForwardAnim.isLastFrame()), thì đặt lại khung hình và chuyển trạng thái sang NOBEHURT, và nếu máu bằng 0, chuyển trạng thái sang FEY.

Trường hợp FEY: Trong trạng thái này, đối tượng đạn chuyển trạng thái sang DEATH.

Trường hợp DEATH: Trong trạng thái chết, không có xử lý nào xảy ra.

Trường hợp NOBEHURT: Trong trạng thái không bị thương, kiểm tra thời gian kể từ khi không bị thương (System.nanoTime() - startTimeNoBeHurt) so với timeForNoBeHurt. Nếu thời gian đã vượt quá timeForNoBeHurt, chuyển trạng thái sang ALIVE.

```

public void drawBoundForCollisionWithMap(Graphics2D g2){
    Rectangle rect = getBoundForCollisionWithMap();
    g2.setColor(Color.BLUE);
    g2.drawRect(rect.x - (int) getGameWorld().camera.getPosX(), rect.y - (int) getGameWorld().camera.getPosY(), rect.width, rect.height);
}
// hàm vẽ va chạm với enemy
public void drawBoundForCollisionWithEnemy(Graphics2D g2){
    Rectangle rect = getBoundForCollisionWithEnemy();
    g2.setColor(Color.RED);
    g2.drawRect(rect.x - (int) getGameWorld().camera.getPosX(), rect.y - (int) getGameWorld().camera.getPosY(), rect.width, rect.height);
}

```

Phương thức `drawBoundForCollisionWithMap()` và `drawBoundForCollisionWithEnemy()` được sử dụng để vẽ ranh giới va chạm của đối tượng đạn với bản đồ và kẻ thù. Dưới đây là mô tả chi tiết của hai phương thức:

Phương thức `drawBoundForCollisionWithMap(Graphics2D g2)`: Phương thức này vẽ ranh giới va chạm của đối tượng đạn với bản đồ. Các bước thực hiện như sau:

Lấy ranh giới va chạm của đối tượng đạn bằng cách gọi phương thức `getBoundForCollisionWithMap()`, và lưu vào biến `rect` kiểu `Rectangle`.

Vẽ một hình chữ nhật bao quanh ranh giới va chạm bằng cách gọi phương thức `drawRect()` của đối tượng `g2` kiểu `Graphics2D`. Các tham số của `drawRect()` là:

`rect.x - (int) getGameWorld().camera.getPosX()`: Tọa độ x của hình chữ nhật tính từ vị trí x của ranh giới va chạm trừ đi vị trí x của camera.

`rect.y - (int) getGameWorld().camera.getPosY()`: Tọa độ y của hình chữ nhật tính từ vị trí y của ranh giới va chạm trừ đi vị trí y của camera.

`rect.width`: Chiều rộng của hình chữ nhật là chiều rộng của ranh giới va chạm.

`rect.height`: Chiều cao của hình chữ nhật là chiều cao của ranh giới va chạm.

Phương thức `drawBoundForCollisionWithEnemy(Graphics2D g2)`: Phương thức này vẽ ranh giới va chạm của đối tượng đạn với kẻ thù. Các bước thực hiện như sau:

Lấy ranh giới va chạm của đối tượng đạn với kẻ thù bằng cách gọi phương thức `getBoundForCollisionWithEnemy()`, và lưu vào biến `rect` kiểu `Rectangle`.

Vẽ một hình chữ nhật bao quanh ranh giới va chạm bằng cách gọi phương thức `drawRect()` của đối tượng `g2` kiểu `Graphics2D`. Các tham số của `drawRect()` tương tự như trong phương thức `drawBoundForCollisionWithMap()`, nhưng sử dụng ranh giới va chạm với kẻ thù (`getBoundForCollisionWithEnemy()`) và màu vẽ là màu đỏ.

2.3.3.16. Class `ParticularObjectManager`

```

public ParticularObject getCollisionWidthEnemyObject(ParticularObject object){
    synchronized(particularObjects){
        for(int id = 0; id < particularObjects.size(); id++){
            ParticularObject objectInList = particularObjects.get(id);

            if(object.getTeamType() != objectInList.getTeamType() &&
                object.getBoundForCollisionWithEnemy().intersects(objectInList.getBoundForCollisionWithEnemy())){
                return objectInList;
            }
        }
    }
    return null;
}

```

Phương thức `getCollisionWidthEnemyObject(ParticularObject object)` được sử dụng để tìm ra đối tượng va chạm với đối tượng khác (`object`) trong danh sách `particularObjects`. Dưới đây là mô tả chi tiết của phương thức:

Sử dụng từ khóa `synchronized` để đồng bộ hóa quá trình truy cập danh sách `particularObjects`. Điều này đảm bảo rằng chỉ có một luồng (thread) có thể truy cập danh sách vào một thời điểm.

Trong từng vòng lặp, lấy đối tượng tại vị trí `id` bằng cách gọi `particularObjects.get(id)` và lưu vào biến `objectInList`.

Kiểm tra nếu `object` và `objectInList` thuộc vào hai đội khác nhau (`object.getTeamType() != objectInList.getTeamType()`) và ranh giới va chạm của chúng giao nhau, thì trả về `objectInList`.

Nếu không tìm thấy đối tượng va chạm thỏa mãn trong danh sách, trả về `null`.

```

public void UpdateObjects() {
    synchronized(particularObjects){
        for(int id = 0; id < particularObjects.size(); id++){

            ParticularObject object = particularObjects.get(id);

            if(!object.isObjectOutOfCameraView()) object.Update();

            if(object.getState() == ParticularObject.DEATH){
                particularObjects.remove(id);
            }
        }
    }

    //System.out.println("Camerawidth = "+camera.getWidth());
}

```

```

public void draw(Graphics2D g2) {
    synchronized(particularObjects){
        for(ParticularObject object: particularObjects)
            if(!object.isObjectOutOfCameraView()) object.draw(g2);
    }
}

```

Phương thức UpdateObjects() được sử dụng để cập nhật các đối tượng trong danh sách particularObjects. Phương thức này kiểm tra nếu đối tượng không ra khỏi tầm nhìn của camera (!object.isObjectOutOfCameraView()), thì gọi phương thức Update() của đối tượng đó để cập nhật.

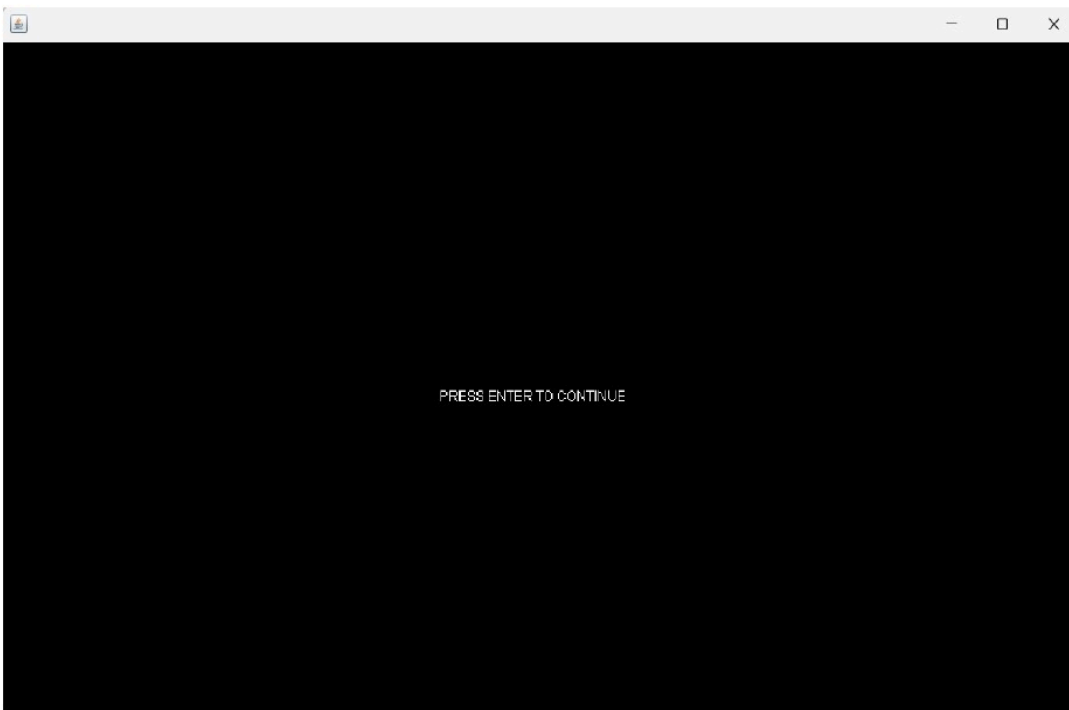
Kiểm tra trạng thái của đối tượng (object.getState()) và nếu đối tượng đã chết (ParticularObject.DEATH), thì loại bỏ đối tượng đó khỏi danh sách bằng cách gọi particularObjects.remove(id).

Phương thức draw(Graphics2D g2) được sử dụng để vẽ các đối tượng trong danh sách particularObjects lên màn hình. Trong từng vòng lặp, lấy đối tượng hiện tại và gọi phương thức isObjectOutOfCameraView() để kiểm tra xem đối tượng có nằm ngoài tầm nhìn của camera hay không. Nếu đối tượng không ra khỏi tầm nhìn của camera, thì gọi phương thức draw(g2) của đối tượng để vẽ nó lên đối tượng g2 kiểu Graphics2D.

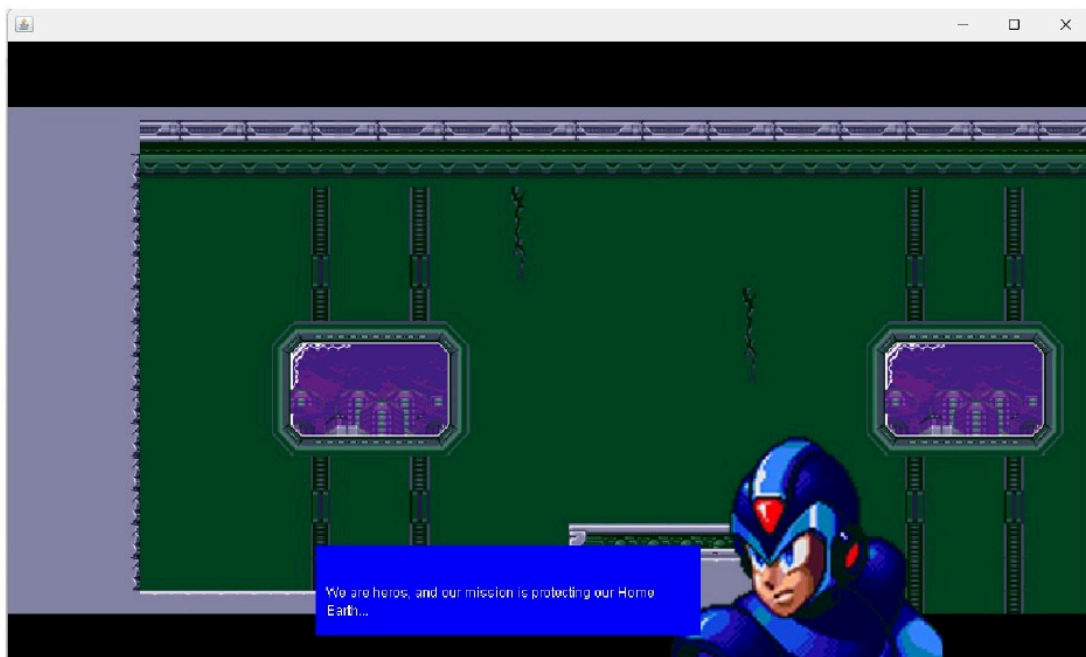
Tổng quan lại, 1 phương thức trên sẽ là kiểm tra xem đối tượng có nằm trong camera hay không, nếu có thì phương thức thứ 2 sẽ vẽ đối tượng đó

III. Một số hình ảnh về game

1. Đoạn mở đầu của game:



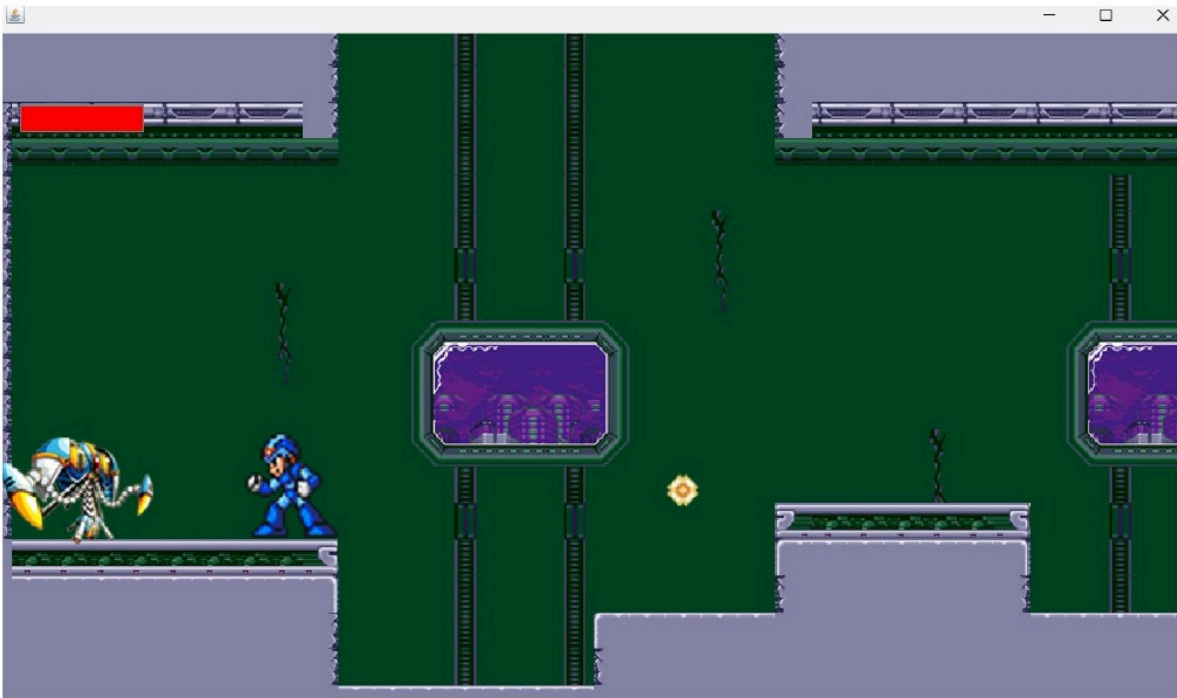
2. Đoạn cutsence và tutorial của game



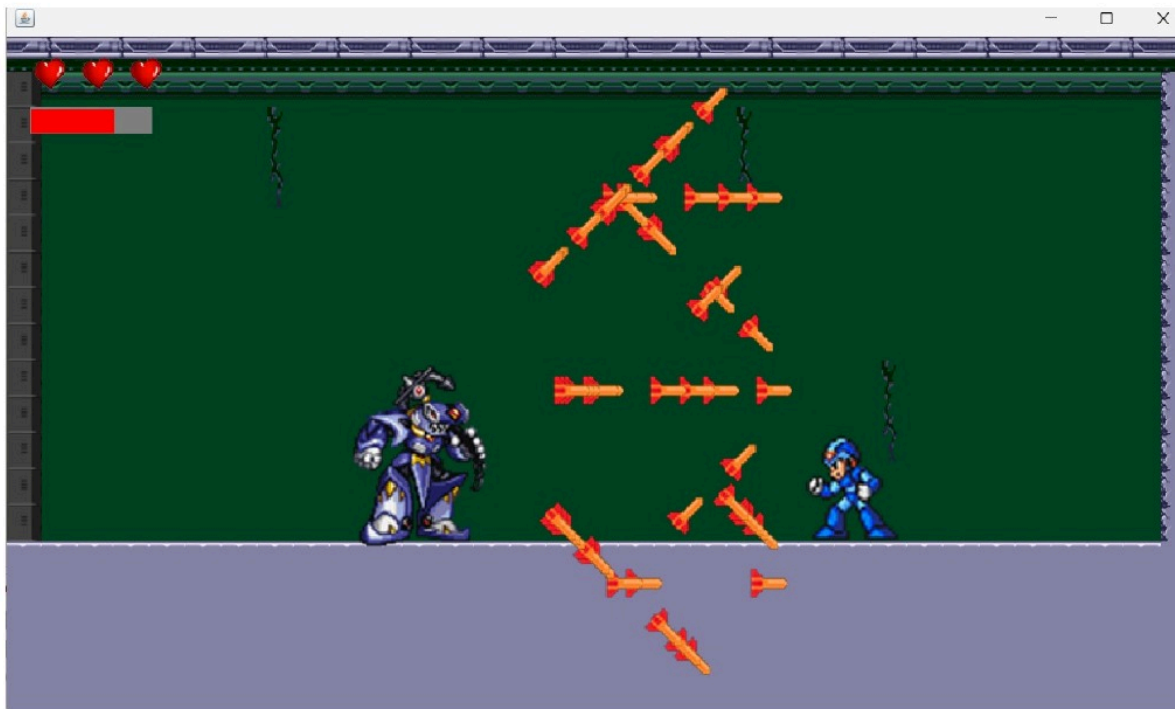
3.Cảnh nhân vật tấn công trong game:



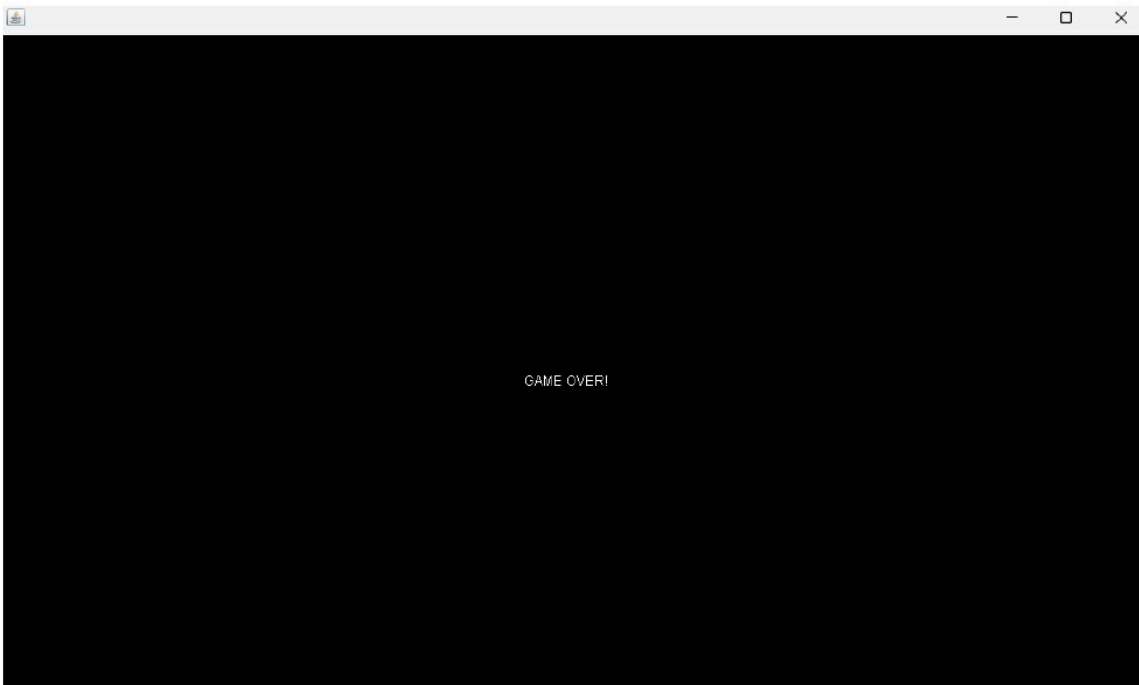
4. Cảnh nhân vật đối đầu với quái trong game:



5. Cảnh gặp boss:



7. Cảnh gameover:





IV. Kết luận

4.1. Kết luận

Bài tập lớn lập trình game Megaman là một dự án thú vị và đầy thách thức. Trong quá trình thực hiện, nhóm đã xây dựng được một game platformer dựa trên nguyên tắc của trò chơi Megaman gốc. Dưới đây là một số điểm nổi bật và kết luận của bài báo cáo:

Xây dựng đồ họa và âm thanh: Đã tạo ra các hình ảnh và hiệu ứng đồ họa để tạo nên một thế giới game hấp dẫn. Đồng thời, âm thanh và nhạc nền được tích hợp để tăng cường trải nghiệm chơi game.

Thiết kế map: Đã thiết kế và xây dựng map chơi đa dạng, với các thử thách và khó khăn khác nhau.

Quản lý đối tượng: Đã triển khai các phương thức và lớp để quản lý các đối tượng trong game, bao gồm nhân vật chính Megaman, quái vật, đạn, vật phẩm và các yếu tố khác. Điều này giúp cho việc xử lý va chạm, di chuyển và tương tác giữa các đối tượng trở nên dễ dàng và linh hoạt.

Xử lý va chạm: Đã triển khai hệ thống xử lý va chạm chính xác và hiệu quả. Bằng cách sử dụng các phương pháp phát hiện va chạm và tính toán hộp va chạm, đã đảm bảo rằng các đối tượng trong game tương tác một cách chính xác và trực quan.

Đa luồng: Để đảm bảo hiệu suất và tránh xung đột, nhóm đã sử dụng các cơ chế đa luồng trong lập trình game. Điều này cho phép xử lý đồng thời các sự kiện và hoạt động trong game mà không làm gián đoạn trải nghiệm chơi.

Đồng bộ hóa: Đã sử dụng các từ khóa đồng bộ hóa như synchronized để đảm bảo an toàn trong truy cập và thay đổi dữ liệu chia sẻ giữa các luồng trong game.

Tổng quan, qua quá trình thực hiện bài tập lớn lập trình game Megaman, nhóm đã có được một trò chơi platformer tương đối hoàn chỉnh, với đồ họa hấp dẫn, gameplay thú vị và tính năng linh hoạt. Qua dự án này, đã rèn luyện kỹ năng lập trình, quản lý dự án và làm việc nhóm. Đồng thời cũng nhận thức được những thách thức và khó khăn trong việc phát triển một trò chơi đáng chú ý như Megaman.

4.2. Tài liệu tham khảo:

<https://youtu.be/YPUSADN1kWg?list=PLRb-vAn1juPRICCFcsZbbOkUYQ8PZn4b>